SECTION 1

Serving with TensorFlow Serving

- **Why serve separately:** A dedicated prediction microservice decouples models from app code, eases versioning, A/B tests, and horizontal scaling, and keeps clients on a stable API (REST or gRPC).
- **SavedModel export:** `tf.saved_model.save()` bundles the computation graph, variables, and signatures; preprocessing should live inside the exported model so training and serving stay aligned.
- **Docker TF Serving:** Containerized serving exposes REST (8501) and gRPC (8500), watches a model repository for new versions, supports graceful version switches, optional batching, and rollback by removing a version folder.

SECTION 2

Cloud deployment (Google AI Platform)

- **Hosted prediction:** Upload SavedModels to Cloud Storage, register model versions on AI Platform, and use autoscaling TF Serving backends with pay-when-used economics.
- **Secure clients:** Service accounts and token-based auth (not end-user credentials) call the REST API via google-api-python-client with JSON instances and `serving_default` signatures.
- **Operations:** Cold-start latency, minimum container counts, load balancing across servers, and Kubernetes orchestration appear when QPS grows.

SECTION 3

Mobile, embedded, and web deployment

- **TensorFlow Lite:** Converts SavedModels to FlatBuffers for low RAM/latency; post-training quantization (8-bit weights) shrinks downloads, while full integer quantization and quantization-aware training preserve speed on device NPUs.
- **Browser inference:** TensorFlow.js loads converted models for on-client prediction when privacy, offline use, or latency matters.
- **Trade-offs:** Smaller bit-widths and fused ops save size and energy at some accuracy cost; edge deployment favors lightweight architectures.

SECTION 4

GPUs and single-machine performance

- **Hardware path:** NVIDIA GPUs with CUDA/cuDNN accelerate training; cloud Deep Learning VMs, Colab, and quotas govern access to accelerators including TPUs.
- **Memory management:** Default full GPU memory capture, CUDA_VISIBLE_DEVICES, virtual device memory limits, and memory growth let multiple processes or notebooks share cards safely.
- **Placement and parallelism:** `tf.device`, soft placement, inter-/intra-op thread pools, and graph scheduling run CPU preprocessing beside GPU math; multiple models or branches can occupy separate devices.

SECTION 5

Training across multiple devices

- **Parallelism choices:** Model parallelism shards layers (often communication-heavy on dense nets); data parallelism replicates the model and averages gradients across replicas—usually preferred.
- **Strategies:** MirroredStrategy (AllReduce on one host), CentralStorageStrategy (parameters on CPU), and comparisons of synchronous vs asynchronous updates address stale gradients and bandwidth saturation.
- **Distribution Strategies API:** `MirroredStrategy().scope()` around model creation trains Keras on all local GPUs with minimal code change; batches must divide evenly across replicas.

SECTION 6

Clusters, cloud training jobs, and tuning

- **Multi-worker training:** TF_CONFIG defines worker/chief/ps/evaluator roles; MultiWorkerMirroredStrategy and ParameterServerStrategy extend mirroring or async ps setups across machines.
- **AI Platform jobs:** `gcloud ai-platform jobs submit training` provisions a TF cluster from packaged trainer code; data on GCS feeds `tf.data` pipelines via `tf.io.gfile`.
- **Hyperparameter search:** Google Vizier (Bayesian optimization) reads TensorBoard accuracy logs from trial output directories to steer parallel training trials efficiently.